our shielding · Your smart contracts, our shielding · Your smart c

shieldify

# Pear Protocol

## Vault

SECURITY REVIEW

Date: 24 December 2025

# CONTENTS


**1. About Shieldify** **3**

**2. Disclaimer** **3**

**3. About Pear Protocol – Vault** **3**

**4. Risk classification** **4**

4.1 Impact 4

4.2 Likelihood 4

**5. Security Review Summary** **4**

5.1 Protocol Summary 4

5.2 Scope 5

**6. Findings Summary** **5**

**7. Findings** **6**

## 1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

## 2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

## 3. About Pear Protocol- Vault

Pear Protocol represents an innovative solution designed to streamline and enhance the efficiency of on-chain pairs trading. It enables users to execute leveraged long and short positions within a single transaction, addressing the complexities and inefficiencies traditionally associated with pair trading in cryptocurrencies. By integrating a variety of on-chain trading engines alongside a dedicated user interface and experience, Pear Protocol simplifies the process of initiating simultaneous long and short positions in correlated assets, such as going long on BTC while shorting ETH with leverage.

This liquidity-agnostic platform offers deep liquidity access and flexibility in managing trading parameters, and mitigates the custody and trust issues found in centralized exchanges by ensuring traders retain asset custody. Beyond simplifying trading executions, Pear Protocol extends its utility with a tokenized trading system, allowing trading positions to be represented as ERC-721 tokens for increased composability within DeFi ecosystems. With features emphasizing simplicity, flexibility, scalability, optionality, and ease of use, Pear Protocol aims to revolutionize pair trading, making it more accessible and efficient while fostering broader decentralized finance trading solutions adoption.

Learn more about Pear's concept and the technicalities behind it here.

### Pear Vault System Summary

This codebase implements the **Pear Vault** system, a decentralized asset management protocol built on Hyperliquid. The system allows whitelisted users (Leaders) to create individual investment vaults. Depositors can then fund these vaults to have the Leader trade with their assets on the Hyperliquid platform.

The architecture is designed to be flexible and upgradeable, utilizing a factory pattern for vault creation and adhering to the `ERC4626` tokenized vault standard.

## 4. Risk Classification

| Severity | Impact: High | Impact: Medium | Impact: Low |
|---|---|---|---|
| **Likelihood: High** | Critical | High | Medium |
| **Likelihood: Medium** | High | Medium | Low |
| **Likelihood: Low** | Medium | Low | Low |

### 4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

### 4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

## 5. Security Review Summary

The security review lasted 10 days with a total of 240 hours dedicated to the audit by three researchers from the Shieldify team.

Overall, the code is well-written. The audit report identified one Critical, one High, seven Medium and fourteen Low severity issues. They're mainly related to broken authorization paths, missing access controls and flawed fund-flow logic that allow arbitrary withdrawals, forced liquidations and unrestricted value transfers between accounts.

This is the third security review that Shieldify has conducted on Pear protocol, with this audit focused primarily on the Vaults functionality.

The Pear team has done a great job with their test suite and provided exceptional support, and promptly implemented the suggested recommendations from the Shieldify researchers.

### 5.1 Protocol Summary

| Project Name | Pear Protocol - Vault |
|---|---|
| **Repository** | pear-vault |
| **Type of Project** | DEX, Vault, ERC4626 |
| **Security Review Timeline** | 10 days |
| **Review Commit Hash** | f681d7cc31ba76d521038f65e0e060849de55345 |
| **Fixes Review Commit Hash** | 5d154f0b8ec4cac6232d3e11e2a37ba0adc2c229 |

### 5.2 Scope

The following smart contracts were in the scope of the security review:

| File | nSLOC |
|---|---|
| src/hyperliquid/L1Read.sol | 264 |
| src/hyperliquid/ICoreWriter.sol | 3 |
| src/libraries/HyperliquidHelper.sol | 55 |
| src/CoreWriterReadCaller.sol | 84 |
| src/PearVaultFactory.sol | 111 |
| src/PearVault.sol | 417 |
| src/Comptroller.sol | 193 |
| **Total** | **1127** |

## 6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Critical** and **High** issues: **2**
- **Medium** issues: **7**
- **Low** issues: **14**
- **Info** issues: **10**

| ID | Title | Severity | Status |
|---|---|---|---|
| [C-01] | Missing Authorization in `withdraw()` and `redeem()` Allows Theft of Any Users Funds | Critical | Fixed |
| [H-01] | Forced Premature Position Liquidation Through Withdrawal Request Manipulation | High | Fixed |
| [M-01] | Withdrawal Griefing Attack via Front-Running | Medium | Fixed |
| [M-02] | Withdrawal Requests Without Share Locking Allow Blocking Instant Redemptions | Medium | Fixed |
| [M-03] | Vault Cannot Be Paused Despite Enum Supporting Paused State | Medium | Fixed |
| [M-04] | Missing Slippage Protection on Vault Operations | Medium | Fixed |

| | | | |
|---|---|---|---|
| [M-05] | In-Flight Transfers from `HyperEVM` to `HyperCore` Not Tracked in `totalAssets()` Causes Share Inflation | Medium | Fixed |
| [M-06] | Withdrawal Request Lacks Recipient Parameter Causing DoS on Blacklist | Medium | Fixed |
| [M-07] | Deposit Limit Bypass via Receiver Manipulation | Medium | Fixed |
| [L-01] | The `deposit(uint256)` Function Unusable Due to Reentrancy Guard Self-Lock | Low | Fixed |
| [L-02] | Excess ETH Not Refunded in `loadFirstDeposit()` | Low | Fixed |
| [L-03] | No Minimum Withdrawal Amount Allows Fee Avoidance | Low | Fixed |
| [L-04] | Vault Configurations Cannot Be Modified or Disabled | Low | Fixed |
| [L-05] | Creation Deposit Fee Has No Upper Bound Validation | Low | Fixed |
| [L-06] | Missing Over-Limit User Enforcement When Updating Deposit Tiers | Low | Fixed |
| [L-07] | Inefficient Capital Transfer Logic in Withdrawal System Disrupts Agent Strategies | Low | Fixed |
| [L-08] | Documentation vs Implementation Mismatch Creates Deceptive Fee Structures | Low | Fixed |
| [L-09] | `PearVault` Is Not ERC4626-Compliant | Low | Fixed |
| [L-10] | Felix Vault Asset Mismatch Not Validated | Low | Fixed |
| [L-11] | Fee Rounding Causes Undercollection by Fee Recipients | Low | Fixed |
| [L-12] | `vaultOwner` Not Updated on Ownership Transfer | Low | Fixed |
| [L-13] | Max Deposit Fails for 18-Decimal Tokens Due to Hard-coded 6-Decimal Assumption | Low | Acknowledged |
| [L-14] | Missing Whitelist Removal Functionality | Low | Fixed |
| [I-01] | Unnecessary Reentrancy Guard in `Comptroller` | Info | Fixed |
| [I-02] | Redundant `\_transferOwnership()` Call in Initialize | Info | Fixed |
| [I-03] | Unused Constants in `Comptroller` | Info | Fixed |
| [I-04] | The `vaultToken` Function Is Redundant | Info | Acknowledged |
| [I-05] | The `getVaultInfo()` Does Not Return `createdAt` Field | Info | Fixed |
| [I-06] | Duplicate and Non-Standard Deposit Event Emitted | Info | Fixed |
| [I-07] | Incorrect Token ID in Comment | Info | Fixed |
| [I-08] | Redundant Zero Check in `requestWithdrawal()` | Info | Fixed |
| [I-09] | Unsafe ERC20 Token Operations | Info | Fixed |
| [I-10] | Use `Ownable2StepUpgradeable` / `Ownable2Step` | Info | Fixed |

## 7. Findings

### [C-01] Missing Authorization in `withdraw()` and `redeem()` Allows Theft of Any User's Funds

#### Severity

Critical Risk

#### Description

The `withdraw()` and `redeem()` functions allow any caller to specify an arbitrary `owner` / `user` address and a `receiver` address. The functions burn shares from the specified owner and send the underlying assets to the receiver without verifying that `msg.sender` is authorized to act on behalf of the owner.

The `_withdrawWithFee()` internal function intentionally bypasses the ERC4626 allowance check by passing the `user` parameter as both `caller` and `owner` to `super._withdraw()`. The base ERC4626 implementation only performs an allowance check when `caller != owner`, so this design completely removes the authorization mechanism.

An attacker can call `redeem(victimShares, attackerAddress, victimAddress)` or `withdraw(assets, attackerAddress, victimAddress)` to burn a victim's shares and receive the assets themselves.

#### Location of Affected Code

File: src/PearVault.sol#L496-L509

```
function redeem(uint256 shares, address receiver, address user) public
    override(ERC4626Upgradeable, IERC4626) nonReentrant onlyActiveVault
    returns (uint256) {
     if (balanceOf(user) < shares) revert InsufficientBalance();

     uint256 assets = super.previewRedeem(shares);
     if(!_canDirectWithdraw(assets)) {
         revert("Direct redeem not allowed. Use requestWithdrawal() for
            queued redeem");
     }

     return _withdrawWithFee(shares, user, receiver);
}
```

File: src/PearVault.sol#L512-L524

```
function withdraw(uint256 assets, address receiver, address owner) public
    override(ERC4626Upgradeable, IERC4626) nonReentrant onlyActiveVault
   returns (uint256) {
    if(!_canDirectWithdraw(assets)) {
        revert("Direct withdraw not allowed. Use requestWithdrawal() for
           queued withdrawal");
    }
    uint256 shares = super.previewWithdraw(assets);
    if (balanceOf(owner) < shares) revert InsufficientBalance();
    _withdrawWithFee(shares, owner, receiver);
    return shares;
}
```

File: src/PearVault.sol#L703-L709

```
function _withdrawWithFee(uint256 shares, address user, address receiver)
    internal returns (uint256) {
  // code
  super._withdraw(
      user, // caller (same as owner to avoid allowance check)
      receiver, // receiver
      user, // owner of shares
      assetsToTransfer, // assets to transfer (after fee)
      shares // shares to burn
  );
  // code
}
```

### Impact

Any user's vault shares can be stolen by any attacker. The attacker specifies themselves as the receiver while targeting any victim's address as the owner, resulting in a complete loss of funds for all vault depositors.

### Recommendation

Add authorization checks at the beginning of both `withdraw()` and `redeem()` functions to ensure that `msg.sender` is either the owner or has sufficient allowance:

```
if (msg.sender != owner) {
    _spendAllowance(owner, msg.sender, shares);
}
```

Alternatively, pass `msg.sender` as the `caller` argument to `super._withdraw()` instead of `user`, allowing the base ERC4626 implementation to enforce the allowance check.

### Team Response

Fixed.

## [H-01] Forced Premature Position Liquidation Through Withdrawal Request Manipulation

### Severity

High Risk

### Description

A critical vulnerability exists in the withdrawal queue system that allows any user to force the backend system to liquidate all trading positions (both spot and perpetual) by requesting withdrawals for amounts equal to or exceeding the entire vault's Total Value Locked (TVL), even when they only own a small fraction of the vault shares.

### Vulnerability Details

The `requestWithdrawal()` function only validates that the user has sufficient shares, but does not validate whether the asset value of those shares is reasonable compared to the vault's liquid capital:

```
function requestWithdrawal(uint256 shares) external override nonReentrant
    validAmount(shares) {
    if (shares == 0) revert InvalidAmount();
    if (balanceOf(msg.sender) < shares) revert InsufficientBalance();  //
        Only checks shares
    if (status != VaultStatus.Active) revert VaultNotActive();

    uint256 withdrawalId = nextWithdrawalId++;

    withdrawalQueue[withdrawalId] = WithdrawalRequest({
        user: msg.sender,
        shares: shares,
        timestamp: block.timestamp,
        executed: false
    });

    userWithdrawalIds[msg.sender].push(withdrawalId);
    totalPendingShares += shares;  //  Can represent more assets than
       vault liquidity

    emit WithdrawalQueued(msg.sender, withdrawalId, shares, block.
       timestamp);
}
```

The Problem: Asset Value vs Share Count The vault's capital is distributed across multiple locations:

- Vault contract balance (liquid)
- Agent wallet balance (liquid)
- Perpetual positions on Hyperliquid (illiquid – requires position closure)
- Spot balances on Hyperliquid (semi-liquid)
- Felix vault deposits (semi-liquid)

When calculating `totalAssets()`:

```
function totalAssets() public view override returns (uint256) {
    uint256 vaultAssetBalance = vaultToken().balanceOf(address(this));
    uint256 agentWalletBalance = IERC20(vaultConfig.assetToken).balanceOf
       (agent);

    if (totalSupply() == 0) {
        return vaultAssetBalance;
    }

    uint256 agentPerpValue = HyperliquidHelper.getAgentPerpValue(agent);
            //  Illiquid
    uint256 agentAssetSpot = HyperliquidHelper.getAgentSpotBalance(...) /
        100; //  Semi-liquid
    uint256 felixBalance = ...; //  Semi-liquid

    return vaultAssetBalance + agentWalletBalance + agentPerpValue +
       agentAssetSpot + felixBalance;
}
```

**The Attack Vector:**

Let's say: - Total vault TVL: $1,000,000 - Vault liquid balance: $50,000 (5%) - Active trading positions: $950,000 (95% - in perp/spot positions) - Attacker owns: 10% of shares = $100,000 worth

**Attack Execution:**

1. Attacker calls `requestWithdrawal()` for their entire 10% share balance and then calls 10 more times.
2. Backend monitoring system sees withdrawal request for $100,0000
3. Backend calculates: Liquid balance ($50,000) < Withdrawal amount ($100,0000)
4. Backend is **forced to close trading positions** worth $50,000+ to fulfill withdrawal
5. **Even if positions are currently at a loss**, they must be closed
6. Vault realizes losses that could have been recovered if positions were held

## Location of Affected Code

File: src/PearVault.sol

## Impact

Trading strategy effectiveness is reduced by forced liquidity maintenance.

## Recommendation

Only allow them to request withdrawal for their shares.

## Team Response

Fixed.

## [M-01] Withdrawal Griefing Attack via Front-Running

### Severity

Medium Risk

### Description

A malicious user can front-run legitimate withdrawal transactions by calling `requestWithdrawal()` to inflate `totalPendingShares`, thereby forcing all subsequent withdrawal attempts to fail and requiring them to use the withdrawal queue instead. This creates a griefing attack vector where a single user can temporarily disable direct withdrawals for all vault participants.

### Vulnerability Details

The `requestWithdrawal()` function does not verify that the vault has sufficient balance to fulfil the withdrawal request at the time of request creation. It only checks:

- The user has sufficient shares
- The vault is active

```
function requestWithdrawal(uint256 shares) external override nonReentrant
    validAmount(shares) {
    if (shares == 0) revert InvalidAmount();
    if (balanceOf(msg.sender) < shares) revert InsufficientBalance();
    if (status != VaultStatus.Active) revert VaultNotActive();

    // ... creates request and increments totalPendingShares
    totalPendingShares += shares;
}
```

### The Attack Vector

The `_canDirectWithdraw()` function blocks direct withdrawals whenever `totalPendingShares > 0`:

```
function _canDirectWithdraw(uint256 amount) internal view returns (bool)
   {
    // Condition 1: No pending withdrawals in queue
    if (totalPendingShares > 0) {
        return false;  //  Blocks ALL direct withdrawals
    }

    // Condition 2: Vault must have sufficient balance for the withdrawal
    return vaultToken().balanceOf(address(this)) >= amount;
}
```

### Attack Scenario

- Setup: Vault has 1,000,000 USDC in balance, supporting direct withdrawals

- Attacker Action: Malicious user front-runs a legitimate withdrawal transaction by calling `requestWithdrawal(1 shares)` – a tiny amount representing minimal value.
- Impact: `totalPendingShares` becomes 1, causing `_canDirectWithdraw()` to return false for ALL users
- Result: All subsequent calls to `withdraw()` and `redeem()` revert with:

"Direct withdrawal not allowed. Use `requestWithdrawal()` for queued withdrawal"

- Persistence: The griefing continues until the attacker's request is executed by `onlyPearAdminAgentWallet()`, which may take time.

### Location of Affected Code

File: src/PearVault.sol#L615-L623

```
function _canDirectWithdraw(uint256 amount) internal view returns (bool)
   {
    // Condition 1: No pending withdrawals in queue
    if (totalPendingShares > 0) {
        return false;  //  Blocks ALL direct withdrawals
    }

    // Condition 2: Vault must have sufficient balance for the withdrawal
    return vaultToken().balanceOf(address(this)) >= amount;
}
```

### Impact

All users must use the withdrawal queue even when the vault has sufficient liquidity

### Recommendation

Add a balance sufficiency check in `requestWithdrawal()`.

### Team Response

Fixed.

## [M-02] Withdrawal Requests Without Share Locking Allow Blocking Instant Redemptions

### Severity

Medium Risk

### Description

When a user calls `requestWithdrawal()`, the withdrawal request is created, but the shares remain with the user and are not locked or transferred to the vault. The `totalPendingShares` counter is incremented to track pending withdrawals.

The `_canDirectWithdraw()` function blocks all instant redemptions (via `redeem()` and `withdraw()`) when `totalPendingShares > 0`. This creates a griefing vector where a malicious user can:

1. Request a withdrawal for their shares
2. Transfer those shares to another address, or create multiple withdrawal requests for the same shares
3. The withdrawal becomes unexecutable because the user no longer holds sufficient shares
4. Only the original requester can cancel via `cancelWithdrawal()`, and they can refuse to do so
5. The `totalPendingShares` remains non-zero, permanently blocking the instant redeem path for all users

In addition to this, `requestWithdrawal()` validates only the amount in the current call against `balanceOf(msg.sender)` and does not account for previously requested but still-pending withdrawals. A user can repeatedly submit requests until the sum of their pending requests greatly exceeds their real holdings, artificially inflating `totalPendingShares`. Since `_canDirectWithdraw()` rejects when `totalPendingShares > 0`, this enables a cheap, scalable denial-of-service against direct withdrawals with minimal capital.

The binary nature of `_canDirectWithdraw()` (requiring `totalPendingShares == 0`) also opens a front-running attack vector. An attacker can monitor the mempool for users attempting direct withdrawals, then front-run by depositing a minimal amount and immediately queuing a tiny withdrawal. This sets `totalPendingShares > 0`, causing the victim's transaction to revert with "Direct withdraw not allowed." The victim is forced into the withdrawal queue despite ample vault liquidity, and the attack can be repeated against multiple targets at minimal cost.

### Location of Affected Code

File: src/PearVault.sol#L465-L493

```
function requestWithdrawal(uint256 shares) external override nonReentrant
    validAmount(shares) {
    if (shares == 0) revert InvalidAmount();
    if (balanceOf(msg.sender) < shares) revert InsufficientBalance();
    if (status != VaultStatus.Active) revert VaultNotActive();

    uint256 withdrawalId = nextWithdrawalId++;

    // Create withdrawal request - shares stay with user
    withdrawalQueue[withdrawalId] = WithdrawalRequest({
        user: msg.sender,
        shares: shares,
        timestamp: block.timestamp,
        executed: false
    });

    userWithdrawalIds[msg.sender].push(withdrawalId);

    // No need to transfer shares - they stay with user until execution
    totalPendingShares += shares;
    // code
}
```

File: src/PearVault.sol#L615-L623

```
function _canDirectWithdraw(uint256 amount) internal view returns (bool)
   {
    // Condition 1: No pending withdrawals in queue
    if (totalPendingShares > 0) {
        return false;
    }

    // Condition 2: Vault must have sufficient balance for the withdrawal
    return vaultToken().balanceOf(address(this)) >= amount;
}
```

The `withdraw()` and `redeem()` functions revert when `_canDirectWithdraw()` returns false:

File: src/PearVault.sol

```
function withdraw(uint256 assets, address receiver, address owner) public
    override nonReentrant onlyActiveVault returns (uint256) {
    if(!_canDirectWithdraw(assets)) {
        revert("Direct withdraw not allowed. Use requestWithdrawal() for
           queued withdrawal");
    }
    // code
}

function redeem(uint256 shares, address receiver, address user) public
   override nonReentrant onlyActiveVault returns (uint256) {
    uint256 assets = super.previewRedeem(shares);
    if(!_canDirectWithdraw(assets)) {
        revert("Direct redeem not allowed. Use requestWithdrawal() for
           queued redeem");
    }
    // code
}
```

### Impact

A malicious user can permanently block the instant redeem functionality for all vault users at minimal cost. The vault owner has no mechanism to cancel fraudulent withdrawal requests, leaving the vault in a degraded state where only queued withdrawals are possible.

Additionally, because `requestWithdrawal()` ignores already-pending amounts, a user can inflate `totalPendingShares` far beyond their real balance (e.g., 1 share can be used to create hundreds of requests). Since `_canDirectWithdraw()` gates exclusively on `totalPendingShares > 0`, direct withdrawals are reliably and cheaply DoS-able even when the vault has ample liquidity.

The binary check also enables targeted front-running: attackers can selectively force specific users into the withdrawal queue by front-running their transactions with a minimal deposit + withdrawal request. The attack requires as little as the minimum deposit amount (~10 USD) plus gas fees, and can be executed repeatedly against high-value withdrawal attempts.

### Proof of Concept

1. Attacker deposits minimum amount (e.g., receives 10 shares)
2. Attacker calls `requestWithdrawal(10)` 100 times in rapid succession
3. Each request passes balance check ( `10 >= 10` )
4. `totalPendingShares` becomes `10 × 100 = 1,000` while attacker still holds only 10 shares
5. `_canDirectWithdraw()` returns false due to `totalPendingShares > 0`, blocking all direct withdrawals
6. Legitimate user attempts a direct withdrawal; is forced into the queue despite sufficient vault liquidity
7. Admin later tries to execute the attacker's withdrawals; the first may succeed, subsequent ones fail for insufficient shares
8. Unless the contract decrements the inflated `totalPendingShares` for failed executions, the value remains elevated, continuing to block direct withdrawals

**Front-running scenario:**

1. Victim prepares a transaction to withdraw 100,000 USD via direct withdrawal
2. Attacker monitors the mempool and detects the victim's pending transaction
3. Attacker front-runs with: (a) deposit 10 USD to receive shares, (b) `requestWithdrawal(10)`
4. `totalPendingShares` becomes 10 (> 0)
5. Victim's transaction executes but reverts because `_canDirectWithdraw()` returns false
6. Victim is forced into the withdrawal queue despite sufficient vault liquidity
7. Attacker can repeat against multiple victims at minimal cost (~10 USD + gas per attack)

### Recommendation

- Lock shares at request time to prevent double-spending and post-request transfers.
- Validate per-user available shares (balance minus pending) and cap new requests accordingly, decrement pending on cancel/execute.
- Add admin/operator ability to cancel stale or unexecutable requests; consider expiry windows.
- Revisit direct-withdrawal gating to avoid blanket blocks based solely on inflated pending values.

### Team Response

Fixed.

## [M-03] Vault Cannot Be Paused Despite Enum Supporting Paused State

### Severity

Medium Risk

### Description

The `VaultStatus` enum defines three states: `Active`, `Paused`, and `Migrating`. The `onlyActiveVault()` modifier checks the status before allowing deposits and withdrawals. However, there is no function to change the vault status after initialization.

The status is set to `Active` during initialization and remains unchangeable for the lifetime of the vault. The `Paused` and `Migrating` states are defined but unreachable.

### Location of Affected Code

File: src/interfaces/IPearVault.sol#L45-L49

```
enum VaultStatus {
    Active,
    Paused,
    Migrating
}
```

File: src/PearVault.sol#L165

```
function _initializeContracts(address vaultOwner__, address agent_,
   string memory name_, address comptroller_) internal {
  // code
  status = VaultStatus.Active;
  // code
}
```

### Impact

The vault owner cannot pause operations in case of an emergency, a detected exploit, or a required migration. Without the ability to halt deposits and withdrawals, any incident response is limited to actions that do not require stopping user interactions with the vault.

### Recommendation

Add a function to allow the owner to change the vault status:

```
function setVaultStatus(VaultStatus newStatus) external onlyOwner {
    VaultStatus oldStatus = status;
    status = newStatus;
    emit VaultStatusChanged(oldStatus, newStatus);
}
```

### Team Response

Fixed.

## [M-04] Missing Slippage Protection on Vault Operations

### Severity

Medium Risk

### Description

The vault's `deposit()`, `mint()`, `withdraw()` and `redeem()` functions lack slippage protection parameters, exposing users to potentially significant losses due to NAV changes.

The vault's `totalAssets()` includes the agent's perpetual trading account value, which can fluctuate significantly due to:

1. Perp position profits/losses
2. Market volatility
3. Liquidations

For queued withdrawals, there is an additional time delay between `requestWithdrawal()` and `executeWithdrawal()`. During this period, the share-to-asset conversion rate can change substantially, but users have no way to specify acceptable bounds.

Affected functions:

1. `deposit()` - no `minSharesOut` parameter
2. `mint()` - no `maxAssetsIn` parameter
3. `withdraw()` - no `maxSharesIn` parameter
4. `redeem()` - no `minAssetsOut` parameter
5. `requestWithdrawal()` - no `minAssetsOut` parameter for queued execution

### Location of Affected Code

File: src/PearVault.sol

### Impact

Users may receive significantly fewer shares on deposit or fewer assets on withdrawal than expected, with no ability to reject unfavourable rates. This is especially concerning for queued withdrawals where execution timing is controlled by the admin.

### Recommendation

Add additional functions with slippage protection parameters as mentioned in the specification:

> If implementors intend to support EOA account access directly, they should consider adding an additional function call for `deposit()` / `mint()` / `withdraw()` / `redeem()` with the means to accommodate slippage loss or unexpected `deposit()` / `withdrawal()` limits, since they have no other means to revert the transaction if the exact output amount is not achieved.

eip-4626#security-considerations

### Team Response

Fixed.

## [M-05] In-Flight Transfers from `HyperEVM` to `HyperCore` Not Tracked in `totalAssets()` Causes Share Inflation

### Severity

Medium Risk

### Description

The `transferToHyperCore()` function transfers vault assets to the HyperCore system via the Hyperliquid bridge. According to the Hyperliquid interaction timings documentation, transfers between HyperEVM and HyperCore are not instant:

- L1 (HyperCore) reads HyperEVM state once per L1 block
- After a `Transfer` event is emitted to the system address, the tokens are credited to HyperCore in the next L1 block
- The in-flight window is 1 block

The `totalAssets()` function calculates the vault's total value by summing: 1. Vault's asset token balance 2. Agent wallet balance 3. Agent's HyperCore perp account value 4. Agent's HyperCore spot balance 5. Felix's vault balance

When `transferToHyperCore()` is called, the assets leave the vault immediately but do not appear in the agent's HyperCore spot balance until the bridge transfer completes. During this in-flight period, the assets are not accounted for in any of the components used by `totalAssets()`, causing a temporary but exploitable reduction in the reported total assets.

This allows a depositor to mint more shares than their deposit warrants, as the share calculation uses the deflated `totalAssets()` value. When the bridge transfer completes and assets reappear in HyperCore, the share price normalizes, diluting existing shareholders.

### Location of Affected Code

File: src/PearVault.sol#L547-L561

```
function transferToHyperCore(uint256 amount) external override onlyOwner
    nonReentrant validAmount(amount) {
     if (vaultToken().balanceOf(address(this)) < amount) revert
        InsufficientBalance();

     // Transfer to agent wallet using the SendFundsToSystem pattern
     _sendFundsToSystem(
         address(vaultToken()),
         agent,
         vaultConfig.systemAddress,
         amount
     );

     emit FundsTransferredToSystem(amount);
}
```

File: src/PearVault.sol#L234-L276

```
function totalAssets() public view override(ERC4626Upgradeable, IERC4626)
    returns (uint256) {
    // 1. Vault asset token balance (6 decimals)
    uint256 vaultAssetBalance = vaultToken().balanceOf(address(this));

    // 2. Agent wallet balance (6 decimals)
    uint256 agentWalletBalance = IERC20(vaultConfig.assetToken).balanceOf
       (agent);

    // ... additional components ...

    // 3. Agent perp account value (6 decimals)
    uint256 agentPerpValue = HyperliquidHelper.getAgentPerpValue(agent);

    // 4. Agent asset token spot balance →(86 decimals)
    uint256 agentAssetSpot = HyperliquidHelper.getAgentSpotBalance(
        agent,
        vaultConfig.assetTokenId
    ) / 100;

    // code
}
```

### Impact

Any deposit executed in the same block after `transferToHyperCore()` is called will use the deflated `totalAssets()` value for share calculations, resulting in inflated share amounts. When the transfer completes in the next block and assets are credited to HyperCore, the share price normalizes, diluting existing shareholders proportionally to the in-flight amount and the new deposit size.

### Recommendation

Track in-flight bridge transfers and include them in the `totalAssets()` calculation. This article provides an example of tracking in-flight transfers implementation: here

### Team Response

Fixed.

## [M-06] Withdrawal Request Lacks Recipient Parameter Causing DoS on Blacklist

### Severity

Medium Risk

### Description

The `requestWithdrawal()` function only accepts a `shares` parameter and automatically sets the requester ( `msg.sender` ) as both the owner and recipient of the withdrawal. The

`WithdrawalRequest` struct does not include a separate receiver field, and when the withdrawal is executed, assets are transferred directly to `request.user`.

If the user's address becomes blacklisted by the asset token contract (USDC and USDHL implement blacklist mechanisms), the asset transfer will revert, making the withdrawal request permanently unexecutable. Since `totalPendingShares` is only decremented upon successful execution or user-initiated cancellation, the unexecutable request blocks the instant redeem path for all vault users.

### Location of Affected Code

File: src/PearVault.sol#L465-L493

```
function requestWithdrawal(uint256 shares) external override nonReentrant
    validAmount(shares) {
    // code
    withdrawalQueue[withdrawalId] = WithdrawalRequest({
        user: msg.sender,
        shares: shares,
        timestamp: block.timestamp,
        executed: false
    });
    // code
}
```

File: src/PearVault.sol#L659-L677

```
function _withdraw(uint256 withdrawalId) internal {
    WithdrawalRequest storage request = withdrawalQueue[withdrawalId];

    if (request.user == address(0)) revert InvalidWithdrawal();
    if (request.executed) revert AlreadyExecuted();

    request.executed = true;
    totalPendingShares -= request.shares;

    uint256 assetsToTransfer = _withdrawWithFee(request.shares, request.
       user, request.user);
    // code
}
```

### Impact

A blacklisted user's pending withdrawal becomes permanently stuck. This blocks the instant redeem functionality for all other vault users since `totalPendingShares` remains non-zero.

### Recommendation

- Add a `receiver` parameter to the `requestWithdrawal()` function and store it in the `WithdrawalRequest` struct:

```
struct WithdrawalRequest {
    address user;
    address receiver;
    uint256 shares;
    uint256 timestamp;
    bool executed;
}

function requestWithdrawal(uint256 shares, address receiver) external
   override nonReentrant validAmount(shares) {
    // code
    withdrawalQueue[withdrawalId] = WithdrawalRequest({
        user: msg.sender,
        receiver: receiver,
        shares: shares,
        timestamp: block.timestamp,
        executed: false
    });
    // code
}
```

Update `_withdraw()` to send assets to `request.receiver` instead of `request.user`.

- Additionally, add the possibility for the admin to cancel withdrawal requests forcibly

### Team Response

Fixed.

## [M-07] Deposit Limit Bypass via Receiver Manipulation

### Severity

Medium Risk

### Description

The `deposit()` limit enforcement mechanism can be completely bypassed by manipulating the receiver parameter in the `deposit()` and `mint()` functions. A user can exceed their tier-based deposit limits by depositing on behalf of other addresses (including fresh addresses they control), while the shares are minted to those addresses, effectively circumventing the investment cap restrictions.

### Root Cause

The deposit limit check is performed on the receiver address rather than the `msg.sender` (the actual depositor/investor):

```
function deposit(uint256 assets, address receiver) public override
   returns (uint256) {
    if (!firstDepositLoaded) revert FirstDepositNotLoaded();
    if (assets < comptroller.getMinimumDeposit()) revert InvalidAmount();

    //  Checks receiver's limit, not msg.sender's limit
    if (receiver != vaultOwner) {
        uint256 maxAllowed = comptroller.getMaxAllowedInvestment(receiver
           );
        if (userDepositAmount[receiver] + assets > maxAllowed) {
            revert DepositLimitExceeded();
        }
    }

    uint256 shares = super.deposit(assets, receiver);

    //  Tracks deposit for receiver, not msg.sender
    userDepositAmount[receiver] += assets;

    emit Deposit(receiver, assets, shares);
    return shares;
}
```

The same vulnerability exists in the `mint()` function.

Who pays vs. Who receives are different entities:

- `msg.sender` provides the funds (the actual investor)
- `receiver` gets the shares (can be any address)
- Limit checking happens on `receiver`, not the fund provider

### Location of Affected Code

File: src/PearVault.sol#L401-L431

```
function deposit(uint256 assets, address receiver) public override
   returns (uint256) {
    if (!firstDepositLoaded) revert FirstDepositNotLoaded();
    if (assets < comptroller.getMinimumDeposit()) revert InvalidAmount();

    //  Checks receiver's limit, not msg.sender's limit
    if (receiver != vaultOwner) {
        uint256 maxAllowed = comptroller.getMaxAllowedInvestment(receiver
           );
        if (userDepositAmount[receiver] + assets > maxAllowed) {
            revert DepositLimitExceeded();
        }
    }

    uint256 shares = super.deposit(assets, receiver);

    //  Tracks deposit for receiver, not msg.sender
    userDepositAmount[receiver] += assets;

    emit Deposit(receiver, assets, shares);
    return shares;
}
```

### Impact

The entire purpose of tier-based investment limits is nullified.

### Recommendation

Track and Limit the Depositor, not Receiver.

### Team Response

Fixed.

## [L-01] The `deposit(uint256)` Function Unusable Due to Reentrancy Guard Self-Lock

### Severity

Low Risk

### Description

The `deposit(uint256 amount)` function is marked with the `nonReentrant` modifier and internally calls `deposit(uint256 assets, address receiver)`, which is also marked with `nonReentrant`. When the first function acquires the reentrancy lock and then calls the second function, the second function attempts to acquire the same lock, triggering a `ReentrancyGuardReentrantCall` error.

This makes the single-parameter `deposit()` function completely unusable.

### Location of Affected Code

File: src/PearVault.sol#L394-L398

```
function deposit(uint256 amount) external override nonReentrant
    onlyActiveVault validAmount(amount) {
     deposit(amount, msg.sender);
}
```

File: src/PearVault.sol#L402-L431

```
 // ERC4626 deposit function
function deposit(uint256 assets, address receiver) public override(
    ERC4626Upgradeable, IERC4626) nonReentrant onlyActiveVault returns (
    uint256){
     if (!firstDepositLoaded) revert FirstDepositNotLoaded();
     if (assets < comptroller.getMinimumDeposit()) revert InvalidAmount();

     // Check deposit limit based on user's tier (vault owner is exempt)
     if (receiver != vaultOwner) {
         uint256 maxAllowed = comptroller.getMaxAllowedInvestment(receiver
             );
         if (userDepositAmount[receiver] + assets > maxAllowed) {
             revert DepositLimitExceeded();
         }
     }

     // Let ERC4626 handle the rest (transfer + mint + event)
     uint256 shares = super.deposit(assets, receiver);

     // Track deposit amount for receiver
     userDepositAmount[receiver] += assets;

     emit Deposit(receiver, assets, shares);
     return shares;
}
```

### Impact

Users cannot deposit funds using the `deposit(uint256)` convenience function. While the two-parameter `deposit(uint256, address)` function remains functional, this breaks the expected interface and may cause integration issues with external contracts or frontends that rely on the single-parameter signature.

### Proof of Concept

Running the existing test with verbose output demonstrates the issue:

```
$ forge test --match-test test_DepositUSDCNative_InsufficientAllowance -
    vvvv
```

```
[17465] 0xfaC50d4b0d5943b5D1E6482Fae08b48F4DF1A6aD::deposit(1000000000 [1
   e9])
   [2300] UpgradeableBeacon::implementation() [staticcall] ←
       [Return] PearVault: [0xe95fEFbaa79748B66DEfb3D662A12541e4d5Cdc8]
   [7763] PearVault::deposit(1000000000 [1e9]) [delegatecall] ←
       [Revert] ReentrancyGuardReentrantCall() ←
    [Revert] ReentrancyGuardReentrantCall()
```

### Recommendation

Remove the `nonReentrant` modifier from the `deposit(uint256 amount)` function since the internal call to `deposit(uint256, address)` already provides reentrancy protection:

```
function deposit(uint256 amount) external override onlyActiveVault
   validAmount(amount) {
    deposit(amount, msg.sender);
}
```

### Team Response

Fixed.

## [L-02] Excess ETH Not Refunded in `loadFirstDeposit()`

### Severity

Low Risk

### Description

The `loadFirstDeposit()` function requires a minimum ETH amount for the creation fee, but does not refund any excess ETH sent by the caller. The check `msg.value < creationFeeAmount` ensures sufficient ETH is provided, but only `creationFeeAmount` is forwarded to the fee recipient. Any excess remains locked in the vault contract.

### Location of Affected Code

File: src/PearVault.sol#L186-L195

```
function loadFirstDeposit(uint256 amount) external payable nonReentrant {
  // code
  // Verify correct native token amount sent
  if (msg.value < creationFeeAmount) revert InsufficientCreationFee();

  // Transfer native token fee to fee recipient
  if (creationFeeAmount > 0) {
      address feeRecipient = comptroller.getFeeRecipient();
      (bool success, ) = payable(feeRecipient).call{value:
         creationFeeAmount}("");
      if (!success) revert InvalidAmount();

      emit CreationFeeCollected(vaultOwner, creationFeeAmount,
         feeRecipient);
  }
  // code
}
```

### Impact

Vault owners who send more ETH than required lose the excess amount permanently in the vault contract.

### Recommendation

Refund excess ETH to the caller:

```
if (msg.value > creationFeeAmount) {
    (bool refundSuccess, ) = payable(msg.sender).call{value: msg.value -
       creationFeeAmount}("");
    require(refundSuccess, "Refund failed");
}
```

### Team Response

Fixed.

## [L-03] No Minimum Withdrawal Amount Allows Fee Avoidance

### Severity

Low Risk

### Description

There is no minimum withdrawal amount enforced for the direct `redeem()` and `withdraw()` functions. When the withdrawal fee is calculated via `_feeOnTotal`, sufficiently small withdrawal amounts result in zero fee due to integer division rounding down.

The fee calculation formula is:

```
feeAmount = (assets * feeBasisPoints) / (feeBasisPoints + 10000)
```

For the fee to round down to zero:

```
assets <= 10000 / feeBasisPoints
```

With the default withdrawal fee of 100 basis points (1%), any withdrawal of 100 asset units or less results in zero fee. For 6-decimal tokens like USDC/USDHL, this equals 0.0001 tokens per withdrawal.

Users can exploit this by splitting a large withdrawal into many small withdrawals, each at or below 100 units, effectively paying zero fees. This can be further optimized by deploying a smart contract that loops small withdrawals in a single transaction, reducing per-withdrawal gas overhead.

Currently, this is not economically profitable given: - Native token (HYPE) price ~$33 USD - Gas price ~1 gwei - Withdrawal operation costs ~54,040 gas per our tests - Cost per withdrawal: ~54,040 × 1 gwei × $33  $0.0018

However, this may become exploitable if: - Withdrawal fee percentage decreases - HYPE token price falls significantly - Gas prices decrease

### Location of Affected Code

File: src/PearVault.sol#L496-L509

File: src/PearVault.sol#L512-L524

### Recommendation

Consider adding a minimum withdrawal amount check to prevent fee avoidance through small withdrawals.

### Team Response

Fixed.

## [L-04] Vault Configurations Cannot Be Modified or Disabled

### Severity

Low Risk

### Description

The `Comptroller` contract provides a `createVaultConfig()` function to add new vault configurations, but there are no functions to update, disable, or delete existing configurations. Once a configuration is created, it remains active and usable for new vault creation indefinitely.

The `VaultConfig` struct contains critical parameters, including asset token address, system address, and Felix vault address. If any of these addresses become deprecated, compromised, or need to be updated, the existing configuration cannot be modified.

### Location of Affected Code

File: src/Comptroller.sol#L402-L413

```
function createVaultConfig(IVaultStructs.VaultConfig memory config)
    external onlyOwner returns (uint256) {
     require(config.assetToken != address(0), "Invalid asset token");
     require(config.systemAddress != address(0), "Invalid system address")
        ;
     require(bytes(config.assetName).length > 0, "Empty asset name");
     // felixVault can be address(0) if not used

     uint256 configId = _vaultConfigs.length;
     _vaultConfigs.push(config);

     emit VaultConfigCreated(configId, config.assetToken, config.assetName
        );
     return configId;
}
```

### Impact

1. Configurations with deprecated or compromised addresses cannot be disabled
2. Whitelisted users can continue creating vaults using problematic configurations
3. No ability to deprecate old configurations when new ones are preferred

### Recommendation

Add functions to update and disable vault configurations:

```
function updateVaultConfig(uint256 configId, IVaultStructs.VaultConfig
    memory config) external onlyOwner {
     require(configId < _vaultConfigs.length, "Invalid config ID");
     _vaultConfigs[configId] = config;
     emit VaultConfigUpdated(configId);
}
```

```
function setVaultConfigEnabled(uint256 configId, bool enabled) external
    onlyOwner {
     require(configId < _vaultConfigs.length, "Invalid config ID");
     _vaultConfigEnabled[configId] = enabled;
     emit VaultConfigStatusChanged(configId, enabled);
}
```

### Team Response

Fixed.

## [L-05] Creation Deposit Fee Has No Upper Bound Validation

### Severity

Low Risk

### Description

The `setCreationDepositFee()` function allows the owner to set an arbitrary fee value without any upper bound validation. A `MAX_CREATION_DEPOSIT` constant is defined as 1000 basis points (10%) but is never used to validate the fee.

### Location of Affected Code

File: src/Comptroller.sol#L61

```
uint256 public constant MAX_CREATION_DEPOSIT = 1000; // 10% maximum
```

File: src/Comptroller.sol#L234-L241

```
function setCreationDepositFee(uint256 newFee) external override
   onlyOwner {
    uint256 oldFee = _creationDepositFee;
    _creationDepositFee = newFee;

    emit CreationDepositFeeUpdated(oldFee, newFee);
}
```

### Impact

The owner can set a fee exceeding the intended 10% maximum, potentially charging vault creators excessively.

### Recommendation

Add validation against the maximum constant:

```
function setCreationDepositFee(uint256 newFee) external override
   onlyOwner {
    require(newFee <= MAX_CREATION_DEPOSIT, "Fee exceeds maximum");
    uint256 oldFee = _creationDepositFee;
    _creationDepositFee = newFee;
    emit CreationDepositFeeUpdated(oldFee, newFee);
}
```

### Team Response

Fixed.

## [L-06] Missing Over-Limit User Enforcement When Updating Deposit Tiers

### Severity

Low Risk

### Description

The `updateDepositTier()` function in the Comptroller contract lacks mechanisms to handle existing users who exceed newly established deposit limits after tier updates. When administrators modify deposit tiers to implement more conservative risk parameters, users who previously deposited amounts under the old limits may suddenly find themselves over the new, lower limits with no enforcement mechanism.

The system only performs deposit limit checks during new deposit transactions but does not monitor or enforce limits for existing positions. This creates a permanent state of regulatory non-compliance and risk concentration, as over-limit users can maintain their positions indefinitely without any requirement to reduce exposure to comply with updated risk management policies.

### Location of Affected Code

File: src/Comptroller.sol#L344-L377

```
function updateDepositTier(uint256 index, uint256 minPearBalance, uint256
    maxDepositAllowed) external onlyOwner {
    require(index < _depositTiers.length, "Invalid tier index");
    require(maxDepositAllowed > 0, "Max deposit must be positive");

    // Check if new minPearBalance conflicts with other tiers
    for (uint256 i = 0; i < _depositTiers.length; i++) {
        if (i != index && _depositTiers[i].minPearBalance ==
           minPearBalance) {
            revert("Another tier with this balance exists");
        }
    }

    DepositTier memory oldTier = _depositTiers[index];

    _depositTiers[index] = DepositTier({
        minPearBalance: minPearBalance,
        maxDepositAllowed: maxDepositAllowed
    });

    emit DepositTierUpdated(
        oldTier.minPearBalance,
        oldTier.maxDepositAllowed,
        minPearBalance,
        maxDepositAllowed
    );

    // No mechanism to handle existing users who now exceed the new
       limits
}
```

### Proof of Concept

1. **Initial state**: Deposit tier allows $1,000,000 maximum per user
2. **User A deposits** $800,000 (under limit)
3. **Risk event occurs** prompting admin to tighten limits
4. **Admin calls** `updateDepositTier()` reducing limit to $500,000
5. **User A remains** with $800,000 deposited (now $300,000 over the new limit)
6. **No enforcement** triggers - User A's position remains active

### Impact

This vulnerability allows permanent risk management policy violations where users can maintain deposit positions that exceed newly established conservative limits. The system becomes unable to effectively respond to changing market conditions or regulatory requirements by updating deposit tiers, as existing over-limit positions remain active indefinitely.

### Recommendation

Implement a comprehensive tier update enforcement system, including grace periods for existing over-limit users to reduce their positions, forced reduction mechanisms for non-compliant positions after grace periods expire, and withdrawal-only modes for users exceeding new limits.

### Team Response

Fixed.

## [L-07] Inefficient Capital Transfer Logic in Withdrawal System Disrupts Agent Strategies

### Severity

Low Risk

### Description

The `_withdrawWithFee()` function in the `PearVault` contract implements inefficient capital transfer logic that unnecessarily pulls excessive funds from agent trading strategies. When the contract lacks sufficient balance to fulfil a withdrawal, the function transfers the entire withdrawal amount from the agent wallet instead of only transferring the deficit amount needed.

This results in agent funds being pulled from active trading positions and sitting idle in the contract vault, reducing overall strategy returns and creating capital inefficiency. The system fails to distinguish between deficit coverage and full withdrawal funding, causing unnecessary disruption to agent trading activities and suboptimal capital allocation across the protocol.

### Location of Affected Code

File: src/PearVault.sol#L679-L735

```
function _withdrawWithFee(uint256 shares, address user, address receiver)
    internal returns (uint256) {
    // Calculate assets to transfer before fees
    uint256 assetsBeforeFee = super.previewRedeem(shares);
    if (assetsBeforeFee == 0) revert InvalidAmount();

    // Transfers entire withdrawal amount, not just deficit
    if (vaultToken().balanceOf(address(this)) < assetsBeforeFee) {
        vaultToken().safeTransferFrom(agent, address(this),
            assetsBeforeFee); // Should transfer only deficit
    }

    // Calculate withdrawal fee using _feeOnTotal (deducts from total
        amount)
    uint256 feeAmount = _feeOnTotal(assetsBeforeFee, _exitFeeBasisPoints
        ());

    // Assets to transfer to user (after fee deduction)
    uint256 assetsToTransfer = assetsBeforeFee - feeAmount;

    // ... rest of withdrawal logic continues
}
```

### Impact

This vulnerability causes significant capital inefficiency by unnecessarily pulling agent funds from active trading strategies into idle contract balances. Agent trading capital is disrupted as funds are transferred to cover the entire withdrawal amount rather than just the deficit between the requested withdrawal and the available contract balance.

### Recommendation

Implement deficit-based transfer logic that calculates the exact amount needed from the agent rather than transferring the entire withdrawal amount. Modify the conditional transfer to compute the difference between the withdrawal amount and the current contract balance, transferring only this deficit from the agent.

### Team Response

Fixed.

## [L-08] Documentation vs Implementation Mismatch Creates Deceptive Fee Structures

### Severity

Low Risk

### Description

The Pear Vault system exhibits critical discrepancies between documented fee structures and actual implementation, creating deceptive user experiences and violating the documented economic model. The documentation clearly states a withdrawal fee of 0.5% (50 basis points), but the `Comptroller` contract implementation sets the default withdrawal fee at 1% (100 basis points), representing a 100% increase over documented rates. Furthermore, the documentation describes a creation deposit fee of 5% (percentage-based on deposit amount), while the implementation uses a fixed 0.001 ETH creation fee unrelated to deposit size.

### Location of Affected Code

File: src/Comptroller.sol#L34-L38

```
// In Comptroller.sol - Mismatched withdrawal fee (1% vs documented 0.5%)
uint256 private _defaultWithdrawalFeeBPS = 100; // 1% DEFAULT, NOT 0.5%

// In Comptroller.sol - Mismatched creation fee type (fixed vs percentage
    )
uint256 private _creationDepositFee = 0.001 ether; // FIXED AMOUNT, NOT
    PERCENTAGE
```

### Impact

This documentation-implementation mismatch creates significant user deception, financial harm, and trust erosion in the protocol. Users expecting 0.5% withdrawal fees are charged 1%, representing a 100% increase that directly reduces their investment returns.

### Recommendation

Immediately align documentation with implementation or modify implementation to match documentation. For the withdrawal fee, either update documentation to state 1% default fee or modify Comptroller to set \codex{_defaultWithdrawalFeeBPS = 50 (0.5%)} to match documentation. For the creation fee, implement a percentage-based calculation matching the documented 5% model by adding a `creationFeeBPS` parameter and calculating the fee as `(depositAmount * creationFeeBPS / 10000)`.

### Team Response

Fixed.

## [L-09] `PearVault` Is Not ERC4626-Compliant

### Severity

Low Risk

### Description

The `PearVault` deviates from the ERC4626 standard in ways that may break integrations and violate integrators' expectations.

#### 1. `withdraw()` Does Not Return Requested Asset Amount

The `previewWithdraw()` function correctly calculates the shares needed to receive a specific asset amount after fees. However, the `withdraw()` function uses `super.previewWithdraw(assets)` (parent implementation without fee adjustment) to calculate shares, then applies fees during the actual withdrawal via `_withdrawWithFee()`.

Per ERC4626, calling `withdraw(assets, receiver, owner)` should transfer exactly `assets` worth of underlying tokens to the receiver and burn more shares, if fees exist. Instead, `PearVault` transfers `assets - fee`, contradicting the standard.

#### 2. `maxDeposit()`, `maxMint()`, `maxWithdraw()`, `maxRedeem()` Not Overridden

ERC4626 specifies that max* functions "MUST factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily), it MUST return 0."

`PearVault` has multiple conditions that disable operations:

1. `status != VaultStatus.Active` disables deposits and withdrawals
2. `!firstDepositLoaded` disables deposits
3. `totalPendingShares > 0` disables direct withdrawals
4. Per-user deposit limits from comptroller

The inherited max* functions do not account for these constraints and return non-zero values when operations would actually revert.

### Location of Affected Code

File: src/PearVault.sol#L512-L524

```
function withdraw(uint256 assets, address receiver, address owner) public
    override(ERC4626Upgradeable, IERC4626) nonReentrant onlyActiveVault
   returns (uint256) {
    if(!_canDirectWithdraw(assets)) {
        revert("Direct withdraw not allowed. Use requestWithdrawal() for
           queued withdrawal");
    }
    uint256 shares = super.previewWithdraw(assets);
    if (balanceOf(owner) < shares) revert InsufficientBalance();
    _withdrawWithFee(shares, owner, receiver);
    return shares;
}
```

### Impact

1. Integrating protocols relying on ERC4626 compliance will receive fewer assets than requested from `withdraw()`
2. Integrators checking max* functions before calling `deposit()` / `withdraw()` will encounter unexpected reverts
3. Automated strategies and aggregators may malfunction when interacting with the vault

### Recommendation

1. Fix `withdraw()` to ensure the receiver gets exactly the requested asset amount by calculating shares to include fee compensation
2. Override max* functions to return appropriate values:

### Team Response

Fixed.

## [L-10] Felix Vault Asset Mismatch Not Validated

### Severity

Low Risk

### Description

When calculating `totalAssets()`, the vault includes the value of shares held in the configured Felix vault by calling `convertToAssets()`. However, there is no validation that the Felix vault's underlying asset matches the Pear vault's asset token.

If a Felix vault with a different underlying asset is configured, the `totalAssets()` calculation would sum incompatible asset values, leading to incorrect share pricing.

### Location of Affected Code

File: src/PearVault.sol#L261-L268

```
function totalAssets() public view override(ERC4626Upgradeable, IERC4626)
    returns (uint256) {
  // code
  // 5. Felix vault balance (if configured)
  uint256 felixBalance = 0;
  if (vaultConfig.felixVault != address(0)) {
      uint256 felixShares = IERC4626(vaultConfig.felixVault).balanceOf(
         address(this));
      if (felixShares > 0) {
          felixBalance = IERC4626(vaultConfig.felixVault).convertToAssets
             (felixShares);
      }
  }
  // code
}
```

### Impact

A misconfigured Felix vault with a different underlying asset would corrupt `totalAssets()` calculations, affecting share minting and redemption rates.

### Recommendation

Validate during vault configuration that the Felix vault's underlying asset matches the Pear vault's asset token:

```
if (config.felixVault != address(0)) {
    require(IERC4626(config.felixVault).asset() == config.assetToken, "
       Felix vault asset mismatch");
}
```

### Team Response

Fixed.

## [L-11] Fee Rounding Causes Undercollection by Fee Recipients

### Severity

Low Risk

### Description

When distributing withdrawal fees between the fee recipient and vault owner, the fee amount is divided by 2 using integer division. When the fee amount is odd, one wei is not distributed to the fee recipients.

The flow is:

1. `feeAmount` is deducted from the user's withdrawal
2. `halfFeeAmount = feeAmount / 2` is calculated
3. `halfFeeAmount` is sent to the fee recipient
4. `halfFeeAmount` is sent to the Vault owner

When `feeAmount` is odd, `halfFeeAmount * 2 < feeAmount` by 1 wei. This 1 wei remains in the vault and becomes part of `totalAssets()`, benefiting shareholders rather than fee recipients.

### Location of Affected Code

File: src/PearVault.sol#L720-L732

```
function _withdrawWithFee(uint256 shares, address user, address receiver)
    internal returns (uint256) {
  // code
  if (feeAmount > 0) {
      address feeRecipient = _exitFeeRecipient();
      // transfer half to feeRecipient and half to leader of vault
      uint256 halfFeeAmount = feeAmount / 2;
      vaultToken().safeTransfer(feeRecipient, halfFeeAmount);
      vaultToken().safeTransfer(vaultOwner, halfFeeAmount);
      emit WithdrawalFeeCollected(
          user,
          feeAmount,
          feeRecipient,
          vaultOwner
      );
  }
  return assetsToTransfer;
}
```

### Impact

Fee recipients (protocol fee recipient and vault owner) receive 1 wei less than intended per withdrawal when the fee amount is odd. The uncollected wei accrues to shareholders instead.

### Recommendation

Give the remainder to one of the recipients:

```
uint256 halfFeeAmount = feeAmount / 2;
vaultToken().safeTransfer(feeRecipient, halfFeeAmount);
vaultToken().safeTransfer(vaultOwner, feeAmount - halfFeeAmount);
```

### Team Response

Fixed.

## [L-12] `vaultOwner` Not Updated on Ownership Transfer

### Severity

Low Risk

### Description

The contract maintains two separate owner-related state variables:

1. `owner()` from `OwnableUpgradeable` - used for access control via `onlyOwner` modifier
2. `vaultOwner` - a custom storage variable used for fee distribution and special privileges

During initialization, both are set to the same address. When `transferOwnership()` is called, only the `OwnableUpgradeable` owner is updated. The `vaultOwner` variable remains pointing to the original owner.

Functions that will recognize the **new owner** (use `onlyOwner` modifier):

1. `transferToHyperCore()`
2. `depositToFelixVault()`
3. `withdrawFromFelixVault()`

Functions that will still use the **old owner** address (use `vaultOwner`):

1. `loadFirstDeposit()` - authorization check `msg.sender != vaultOwner`
2. `deposit()` - deposit limit exemption check `receiver != vaultOwner`
3. `_withdrawWithFee()` - fee distribution `vaultToken().safeTransfer(vaultOwner, halfFeeAmount)`

### Location of Affected Code

File: src/PearVault.sol#L178

```
function loadFirstDeposit(uint256 amount) external payable nonReentrant {
  if (msg.sender != vaultOwner) revert NotAuthorized();
  // code
}
```

File: src/PearVault.sol#L416

```
function deposit(uint256 assets, address receiver) public override(
    ERC4626Upgradeable, IERC4626) nonReentrant onlyActiveVault returns (
    uint256) {
  // code
  if (receiver != vaultOwner) {
  // code
}
```

File: src/PearVault.sol#L725

```
function _withdrawWithFee(uint256 shares, address user, address receiver)
    internal returns (uint256) {
  // code
  vaultToken().safeTransfer(vaultOwner, halfFeeAmount);
  // code
}
```

### Impact

After ownership transfer:

1. The new owner cannot call `loadFirstDeposit()`
2. The new owner is subject to deposit limits while the old owner remains exempt
3. Withdrawal fees continue to be sent to the old owner

### Recommendation

Remove the `vaultOwner` storage variable entirely and use `owner()` in all places where `vaultOwner` is currently referenced. The `vaultOwner` variable is redundant since `OwnableUpgradeable` already tracks the owner.

### Team Response

Fixed.

## [L-13] Max Deposit Fails for 18-Decimal Tokens Due to Hardcoded 6-Decimal Assumption

### Severity

Low Risk

### Description

The protocol incorrectly assumes that all underlying ERC-20 assets operate on 6 decimals, while some popular stablecoins use 18 decimals. This mismatch causes the computed maximum deposit limit to be inconsistent with the real token precision.

Example impacted code:

```
uint256 maxAllowed = comptroller.getMaxAllowedInvestment(receiver);
```

The `getMaxAllowedInvestment()` value is defined inside the `Comptroller` contract using a 10^6 (USDC-style) decimal basis. When the vault/token uses 18 decimals, the deposit comparison logic scales incorrectly.

As a result, attempting a "max deposit" of an 18-decimal token will revert even when the user is below the intended limit. And there are other places as well, which can get affected.

### Location of Affected Code

File: src/PearVault.sol#L450

```
function mint(uint256 shares, address receiver) public override(
    ERC4626Upgradeable, IERC4626) nonReentrant onlyActiveVault returns (
    uint256) {
  // code
  uint256 maxAllowed = comptroller.getMaxAllowedInvestment(receiver);
  // code
}
```

File: src/PearVault.sol#L417

```
function deposit(uint256 assets, address receiver) public override(
    ERC4626Upgradeable, IERC4626) nonReentrant onlyActiveVault returns (
    uint256) {
  // code
  uint256 maxAllowed = comptroller.getMaxAllowedInvestment(receiver);
  // code
}
```

### Impact

Max deposit transaction fails for 18-decimal underlying tokens.

### Recommendation

Normalize all boundary checks to asset decimals.

### Team Response

Acknowledged.

## [L-14] Missing Whitelist Removal Functionality

### Severity

Low Risk

### Description

The `PearVaultFactory` contract implements a whitelist mechanism to control who can create vaults, but only provides a function to add users to the whitelist ( `whitelistUser()` ) with no corresponding function to remove them. This creates a permanent, irrevocable privilege escalation where once a user is whitelisted, they cannot be removed even if they become malicious, compromised, or no longer authorized.

### Location of Affected Code

File: src/PearVault.sol

```
mapping(address => bool) private _whitelisted;

modifier onlyWhitelisted() {
    if (!_whitelisted[msg.sender]) revert NotWhitelisted();
    _;
}

//  Function to ADD users to whitelist
function whitelistUser(address user) external onlyOwner {
    if (user == address(0)) revert InvalidAddress();
    _whitelisted[user] = true;
    emit UserWhitelisted(user);
}

//  NO FUNCTION to REMOVE users from whitelist
// Missing: removeFromWhitelist() or dewhitelistUser()

function createVault(address owner_, address agent, string memory name_,
   string memory description, string memory imageURI, uint256 configId)
   external override nonReentrant
    onlyWhitelisted  // ← Uses whitelist check
    returns (address vaultAddress)
{
    // Only whitelisted addresses can create vaults
    // code
}
```

### Impact

Inability to revoke access from compromised accounts, malicious users, or users who no longer need vault creation privileges, leading to permanent security risks and loss of access control.

### Recommendation

Add a Simple Removal Function.

### Team Response

Fixed.

## [I-01] Unnecessary Reentrancy Guard in Comptroller

### Severity

Informational Risk

### Description

The `Comptroller` contract inherits from `ReentrancyGuardUpgradeable` but does not perform any external calls that would require reentrancy protection. All functions are simple getters or setters that modify storage directly.

### Location of Affected Code

File: src/Comptroller.sol

### Recommendation

Remove the `ReentrancyGuardUpgradeable` inheritance to reduce contract size and deployment costs.

### Team Response

Fixed.

## [I-02] Redundant `_transferOwnership()` Call in Initialize

### Severity

Informational Risk

### Description

The `initialize()` function calls `_transferOwnership(vaultOwner__)` after `__Ownable_init(vaultOwner__)`. The `__Ownable_init()` already sets the owner to the provided address, making the subsequent `_transferOwnership()` call redundant.

### Location of Affected Code

File: src/PearVault.sol#L140

### Recommendation

Remove the redundant `_transferOwnership()` call.

### Team Response

Fixed.

## [I-03] Unused Constants in `Comptroller`

### Severity

Informational Risk

### Description

The following constants are defined but never used in the contract:

```
uint256 public constant MAX_MANAGEMENT_FEE = 2000; // 20% maximum
uint256 public constant MAX_CREATION_DEPOSIT = 1000; // 10% maximum
```

### Location of Affected Code

File: src/Comptroller.sol

### Recommendation

Either implement validation using these constants or remove them to reduce contract size.

### Team Response

Fixed.

## [I-04] The `vaultToken` Function Is Redundant

### Severity

Informational Risk

### Description

The `vaultToken()` function simply returns the ERC4626 underlying asset, which is already accessible via the inherited `asset()` function from ERC4626.

### Location of Affected Code

File: src/PearVault.sol#L339-L341

```
function vaultToken() public view returns (IERC20) {
    return IERC20(asset());
}
```

### Recommendation

Use `asset()` directly and remove the redundant wrapper function.

### Team Response

Acknowledged.

## [I-05] The `getVaultInfo()` Does Not Return `createdAt` Field

### Severity

Informational Risk

### Description

The `VaultInfo` struct contains 5 fields, including `createdAt`, but the `getVaultInfo()` function only returns 4 fields (owner, agent, name, vaultToken). The `getVaultDetails()` function returns the complete struct.

This inconsistency between the two getter functions may cause confusion for integrators.

### Location of Affected Code

File: src/PearVaultFactory.sol

### Recommendation

Either add `createdAt` to `getVaultInfo()` return values or document that `getVaultDetails()` should be used for complete information.

### Team Response

Fixed.

## [I-06] Duplicate and Non-Standard Deposit Event Emitted

### Severity

Informational Risk

### Description

The `deposit()` function emits a custom `Deposit` event after calling `super.deposit()`. The parent ERC4626 implementation already emits the standard `Deposit(caller, receiver, assets, shares)` event, resulting in duplicate event emission.

Additionally, the custom event has a non-standard signature with only 3 parameters instead of the ERC4626-specified 4 parameters:

- Standard ERC4626: `Deposit(address indexed caller, address indexed owner, uint256 assets, uint256 shares)`
- PearVault custom: `Deposit(address indexed user, uint256 amount, uint256 shares)`

### Location of Affected Code

File: src/PearVault.sol#L423-L424

```
function deposit(uint256 assets, address receiver ) public override(
    ERC4626Upgradeable, IERC4626) nonReentrant onlyActiveVault returns (
    uint256) {
  // code
  // Let ERC4626 handle the rest (transfer + mint + event)
  uint256 shares = super.deposit(assets, receiver);

  // Track deposit amount for receiver
  userDepositAmount[receiver] += assets;

  emit Deposit(receiver, assets, shares);
}
```

File: src/interfaces/IPearVault.sol#L57

```
event Deposit(address indexed user, uint256 amount, uint256 shares);
```

### Impact

1. Duplicate events for the same deposit action may confuse off-chain indexers
2. Non-standard event signature may break integrations expecting ERC4626-compliant events

### Recommendation

Remove the custom `Deposit` event emission since the parent contract already emits the standard ERC4626 event.

### Team Response

Fixed.

## [I-07] Incorrect Token ID in Comment

### Severity

Informational Risk

### Description

The NatSpec comment for the `spotBalance()` function incorrectly states that token ID 150 corresponds to USDHL. On Hyperliquid mainnet:

- Token ID 150 (0x96) = HYPE
- Token ID 291 (0x123) = USDHL, reference: https://app.hyperliquid.xyz/explorer/token/0xd289c79872a9e

If this comment is followed during deployment configuration, incorrect token IDs may be used.

### Location of Affected Code

File: src/libraries/HyperliquidHelper.sol#L70-L75

```
/**
 * @notice Get spot balance from Hyperliquid precompile
 * @param user User address to query
 * @param token Token ID (0=USDC, 150=USDHL, etc.)
 * @return Spot balance data
 */
```

### Recommendation

Update the comment to reflect correct token IDs:

```
@param token Token ID (0=USDC, 150=HYPE, 291=USDHL)
```

### Team Response

Fixed.

## [I-08] Redundant Zero Check in `requestWithdrawal()`

### Severity

Informational Risk

### Description

The `requestWithdrawal()` function uses the `validAmount(shares)` modifier, which already reverts if `shares == 0`. Line 468 then performs the same check again.

### Location of Affected Code

File: src/PearVault.sol#L465-L468

```
function requestWithdrawal(uint256 shares) external override nonReentrant
    validAmount(shares) {
    if (shares == 0) revert InvalidAmount();
    // code
}
```

### Recommendation

Remove the redundant `if (shares == 0)` check since `validAmount` modifier already handles this.

### Team Response

Fixed.

## [I-09] Unsafe ERC20 Token Operations

### Severity

Informational Risk

### Description

The `sendFundsToSystem()` function uses raw ERC20 `transfer()` and `transferFrom()` calls without checking return values or using OpenZeppelin's SafeERC20 library. This can lead to silent transfer failures with non-standard ERC20 tokens, resulting in state inconsistencies, stuck funds, and potential financial losses.

### Location of Affected Code

File: src/CoreWriterReadCaller.sol#L101-L127

```
function sendFundsToSystem(uint256 amount, address token, address
   backendWallet) external {
    IERC20 tokenContract = IERC20(token);

    require(
        tokenContract.balanceOf(address(this)) >= amount,
        "Insufficient balance"
    );

    //  VULNERABILITY: No return value check
    tokenContract.transfer(backendWallet, amount);

    require(
        tokenContract.allowance(backendWallet, address(this)) >= amount,
        "Insufficient allowance"
    );

    //  VULNERABILITY: No return value check
    tokenContract.transferFrom(
        backendWallet,
        SYSTEM_WALLET_ADDRESS,
        amount
    );
}
```

### Impact

Transfers may silently fail with tokens like USDT, BNB, and OMG, causing the contract to believe tokens were transferred when they actually weren't.

### Recommendation

Consider using safe transfer functions.

### Team Response

Fixed.

## [I-10] Use `Ownable2StepUpgradeable` / `Ownable2Step`

### Severity

Informational Risk

### Description

The `Ownable2Step` pattern is an improvement over the traditional `Ownable` pattern, designed to enhance the security of ownership transfer functionality in a smart contract. Unlike the original `Ownable` pattern, where ownership can be transferred directly to a specified address, the `Ownable2Step` pattern introduces an additional step in the ownership transfer process. Ownership transfer only completes when the proposed new owner explicitly accepts the ownership, mitigating the risk of accidental or unintended ownership transfers to mistyped addresses.

### Location of Affected Code

File: src/PearVault.sol

File: src/Comptroller.sol

File: src/PearVaultFactory.sol

### Impact

Without the `Ownable2Step` pattern, the contract owner might inadvertently transfer ownership to an unintended or mistyped address, potentially leading to a loss of control over the contract. By adopting the `Ownable2Step` pattern, the smart contract becomes more resilient against external attacks aimed at seizing ownership or manipulating the contract's behavior.

### Recommendation

It is recommended to use either `Ownable2Step` or `Ownable2StepUpgradeable`, depending on the smart contract.

### Team Response

Fixed.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

# Thank you!